

CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
Faculdade de Tecnologia de Jundiaí
Curso Superior de Tecnologia em Sistemas Embarcados

PROJETO DE SENSOR DE PRESSÃO PARA PNEUS

Júlio César Lemos Ribeiro - RA 1141372121006

Jundiaí
(2026)

CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA

Faculdade de Tecnologia de Jundiaí

Curso Superior de Tecnologia em Sistemas Embarcados

PROJETO DE SENSOR DE PRESSÃO PARA PNEUS

Trabalho apresentado à Faculdade de Tecnologia de Jundiaí como requisito para aprovação na disciplina de Projeto Integrador II, do curso de Tecnologia em Sistemas Embarcados, sob a orientação da Prof.^a Luciana Ferreira Baptista.

**Jundiaí
(2026)**

Sumário

1 INTRODUÇÃO.....	4
1.1 FINALIDADE DESTE TRABALHO.....	4
1.2 ESCOPO.....	4
2.1 VISÃO GERAL DO SISTEMA.....	5
2.2 MÓDULOS DO SISTEMA.....	5
2.2.1 MÓDULOS FUNCIONAIS.....	5
2.2.2 MÓDULOS DE HARDWARE.....	6
3 DOCUMENTAÇÃO.....	8
3.1 REQUISITOS DO SISTEMA.....	8
3.1.1 REQUISITOS FUNCIONAIS (RF).....	8
3.1.2 REQUISITOS NÃO FUNCIONAIS (RNF).....	8
3.2 DIAGRAMAS UML RELEVANTES.....	9
3.2.1 DIAGRAMA DE CASOS DE USO.....	9
3.3.2 DIAGRAMA DE MÁQUINA DE ESTADOS.....	10
4 MONTAGEM DO SENSOR.....	11
5 INTEGRAÇÃO COM SISTEMA SUPERVISÓRIO.....	12
6 COMUNICAÇÃO ENTRE APLICAÇÃO E USUÁRIO.....	14
6.1 INTERFACE LOCAL (DISPLAY OLED SSD1306).....	14
6.2 INTERFACE REMOTA (WEB BROWSER).....	14
7 CRIAÇÃO DE UM REPOSITÓRIO NO GITHUB.....	15
8 CONCLUSÃO.....	16
9 REFERÊNCIAS BIBLIOGRÁFICAS.....	17

1 INTRODUÇÃO

A manutenção da pressão correta dos pneus é um fator crítico para a segurança veicular, eficiência de combustível e vida útil dos componentes pneumáticos. Sistemas de medição convencionais (manômetros analógicos) carecem de conectividade e histórico, limitando a agilidade em frotas ou oficinas.

1.1 FINALIDADE DESTE TRABALHO

Este trabalho apresenta o desenvolvimento de um manômetro portátil inteligente baseado no microcontrolador ESP32 e desenvolvido sob a framework nativa Espressif IoT Development Framework (ESP-IDF) em linguagem C/C++.

Ele faz parte da entrega da Disciplina Projeto Integrador III do 4º Semestre do Curso de Sistemas Embarcados da Fatec Jundiaí que visa integrar os conhecimentos adquiridos em outras disciplinas do mesmo semestre.

Cabe esclarecer que o aluno responsável pelo projeto já cursou as demais disciplinas no 2º semestre de 2024 e entregou os respectivos trabalhos cobrado por elas na ocasião, podendo haver divergências no material aqui apresentado em relação ao conteúdo anterior, tendo em vista que o projeto proposto anteriormente sofreu alterações em relação ao atual.

Esclareço também que por conta do grau de exigência do escopo de 2024 e da sobrecarga de trabalho + estágio + faculdade, fomos obrigados a abandonar a disciplina de PI III para cursá-la somente agora, para um melhor aproveitamento.

1.2 ESCOPO

Originalmente em 2024 o projeto deveria apresentar um "Produto Mínimo Viável" (MVP) de um manômetro digital portátil para que um usuário pudesse medir a pressão de pneus de carro no padrão de bico Schrader e verificar se o mesmo poderia rodar em segurança.

Esse projeto deveria utilizar um conjunto entregue para cada dupla contendo: uma placa TTGO conjugada com microprocessador ESP32 + display OLED + suporte para Bateria 18650, com a bateria já inclusa. Além de um sensor BMP280 para Temperatura e pressão barométrica, como também um sensor de pressão SMP3011.

Ao ligar o sistema deveria mostrar no display a temperatura e pressão barométrica local e após essa exibição deveria estar apto para medir a pressão dos pneus. Essa medição deveria ser feita em qualquer condição de luminosidade, trabalhando com uma precisão mínima de 5%, apresentando resultados tanto em bar como psi e comparar com valores pré-definidos pelo fabricante do veículo.

Originalmente também havia a exigência de que o produto deveria ser oferecido ao mercado em um preço competitivo na faixa dos R\$ 40 na época e ser apresentado ao *Product Owner* (representantes da empresa Tex) em uma *elevator pitch* de 30 segundos.

Estamos retomando o que havia sido exigido anteriormente, pois nesse semestre por conta de uma falha de comunicação, só fomos informados das palestras com a TEX quando algumas delas já haviam sido realizadas.

Sendo assim, não pudemos participar de todas para saber detalhes de alterações do escopo e naquelas reuniões em que participamos, o instrutor Henrique verbalizou que não haveria a cobrança do desenvolvimento de um MVP com encapsulamento.

Atualmente o escopo que nos foi exigido durante os treinamentos com a empresa TEX é que deveria somente ser desenvolvido um dispositivo capaz de realizar leituras em tempo real de pressão e temperatura dos pneus, onde o foco deveria ser o código e que o mesmo deveria ser compartilhado em um repositório.

Como o código foi entregue pelo instrutor quase pronto, o aluno deveria focar em melhorias como, por exemplo, a compensação das temperaturas nas leituras das pressões e foi com essa meta que trabalhamos.

2 DEFINIÇÃO DO SISTEMA

2.1 VISÃO GERAL DO SISTEMA

O presente dispositivo além de realizar leituras em tempo real de pressão e temperatura de pneus e do ambiente, faz as devidas compensações exigidas e armazena o histórico de múltiplos pneumáticos em uma sessão.

Esses dados são disponibilizados localmente através de uma interface gráfica integrada (Display OLED) e remotamente via Servidor Web embarcado (Wi-Fi), operando de forma autônoma sem dependência de infraestrutura de rede externa, permitindo que dispositivos móveis como celulares, possam visualizar as leituras obtidas.

2.2 MÓDULOS DO SISTEMA

Em Sistemas Embarcados podemos enxergar os Módulos do Sistema sob a ótica dos Módulos de Hardware ou sob a ótica dos Módulos funcionais.

2.2.1 MÓDULOS FUNCIONAIS

Em termos funcionais, podemos agrupar os elementos, sejam de hardware ou software, de acordo com os recursos que oferecem ou pelos processos dos quais participam. Assim temos:

Módulo de Aquisição de Dados:

Responsável pela leitura das variáveis externas e constituído pelos sensores de pressão SMP3011 e BMP280 que adquirem os dados na forma de leituras;

Módulo de Processamento:

Responsável por centralizar o tratamento das informações, que no caso é constituído pelo processador ESP32;

Módulo de Comunicação:

Constituído pelo barramento I2C que permite a comunicação entres sensores e placa e pelo Servidor Web interno montado através de código e bibliotecas que permitem a conexão de clientes externos por protocolo TCP/IP.

Módulo de Exibição:

Responsável pela apresentação dos dados e constituído internamente pelo Display OLED e externamente pelo Front End exibido nos celulares conectados;

2.2.2 MÓDULOS DE HARDWARE

Já em termos de Hardware podemos decompor nosso sistema de acordo com os sensores e componentes utilizados.

Sensor SMP3011:

Sensor de pressão piezoresistivo integrado com condicionamento de sinal interno e interface de comunicação digital I2C. Atua na escala manométrica relativa, fornecendo leituras diretas da pressão interna do pneu e temperatura do fluxo de ar, com compensação térmica de fábrica. Oferece também a possibilidade de leitura de temperatura desse fluxo.



Sensor BMP280:

Sensor barométrico digital absoluto com comunicação I2C. Utilizado para monitorar as condições atmosféricas locais (pressão ambiente e temperatura do ar), servindo como base de calibração e exibição em estado de repouso do dispositivo. Muito utilizado também como altímetro, tendo em vista a relação entre pressão barométrica e altitude.



Display OLED:

Display conjugado a placa principal com resolução 128x64 e tela de 0.96" com comunicação por I2C permitindo o compartilhamento de pinagem com outros sensores.

**Bateria 18650:**

Fornecer 3.7 V nominais permitindo a alimentação do sistema, garantindo sua autonomia independente de fontes externas.

**Plata TTGO ESP32:**

Conjugado com display e bateria mencionados anteriormente, centraliza a capacidade de processamento usando um módulo ESP-WROOM-32 fornecendo conexão via Wifi e Bluetooth



Placa TTGO ESP32 WROOM c/ Display Oled e Bateria conjugados



3 DOCUMENTAÇÃO

3.1 REQUISITOS DO SISTEMA

3.1.1 REQUISITOS FUNCIONAIS (RF)

RF01	O sistema deve monitorar e exibir continuamente a temperatura e a pressão atmosférica ambiente no display enquanto estiver em estado de repouso.
RF02	O sistema deve detectar automaticamente a conexão do bico ao pneu quando a pressão manométrica ultrapassar o limiar de 3.0 kPa.
RF03	O sistema deve capturar e registrar o valor máximo de pressão (pico) atingido durante o processo de medição de cada pneu.
RF04	O sistema deve reter a exibição dos dados do pneu concluído no display por um período fixo de 10 segundos após a desconexão do bico.
RF05	O sistema deve incrementar automaticamente o índice do pneu (P1, P2... P10) ao detectar uma nova conexão de pressão.
RF06	O sistema deve inicializar um ponto de acesso Wi-Fi (Access Point) com o SSID "PROJETO" e hospedar um servidor HTTP.
RF07	O servidor HTTP deve servir uma página HTML dinâmica contendo os dados de ambiente e a tabela de histórico estático com todos os pneus medidos.
RF08	O sistema deve retornar ao estado de repouso ("AGUARDANDO") e limpar todo o histórico de pneus ao receber o comando físico do botão de BOOT (GPIO 0).

3.1.2 REQUISITOS NÃO FUNCIONAIS (RNF)

RNF01	O firmware deve ser desenvolvido na framework nativa ESP-IDF em linguagem C/C++ para garantir o controle de baixo nível e otimização de memória.
RNF02	O sistema operacional de tempo real FreeRTOS deve ser utilizado para o agendamento de tarefas preemptivas e distribuição de carga entre os núcleos da CPU.
RNF03	A renderização gráfica local deve utilizar a biblioteca Light and Versatile Graphics Library (LVGL), rodando de forma isolada e protegida por exclusão mútua (Mutex).
RNF04	O tempo de resposta para a detecção de conexão e desconexão do pneu não deve ultrapassar 500 milissegundos.
RNF05	O servidor web deve ser capaz de processar requisições HTTP sem causar interrupções ou congelamento na amostragem dos sensores (isolamento de Core).

3.2 DIAGRAMAS UML RELEVANTES

3.2.1 DIAGRAMA DE CASOS DE USO

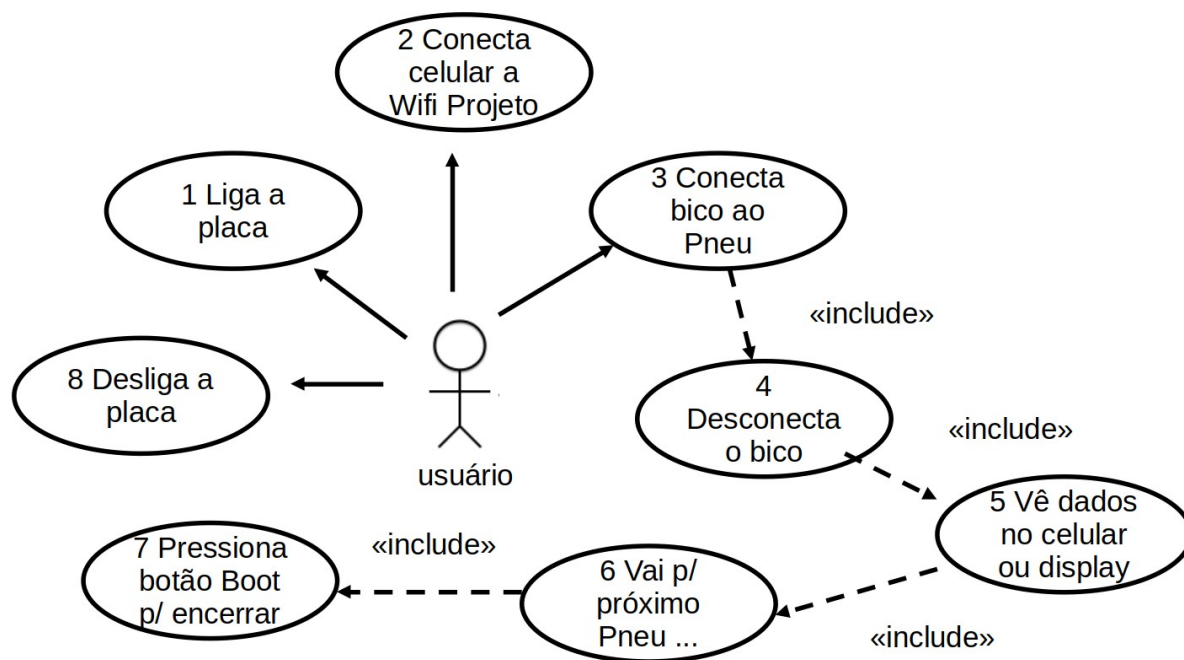


Imagem ilustrando os Casos de Uso

Ao ligar a placa o usuário habilita a rede Wifi Projeto cuja senha para teste é 12345678. Ele deve conectar seu celular a este e acessar pelo navegador o endereço <http://192.168.4.1>.

Em seguida, na versão final do produto, deve conectar o bico do dispositivo à válvula do primeiro pneu por cerca de 10 segundos. Ao desconectar da válvula a leitura estará disponível no display do dispositivo e na tela do celular para sua visualização.

Se desejar verificar outro pneu, deve repetir o procedimento de conectar e desconectar no próximo. Caso queira encerrar o procedimento de leitura, deverá pressionar o botão reset da placa. Isso permite que o dispositivo volte ao estado de aguardando, habilitando o usuário a verificar outro veículo.

Caso queira encerrar definitivamente as leituras, deverá desligar a placa pelo interruptor. A rede Wifi será desfeita e a tela do celular mostrará página não encontrada.

3.3.2 DIAGRAMA DE MÁQUINA DE ESTADOS

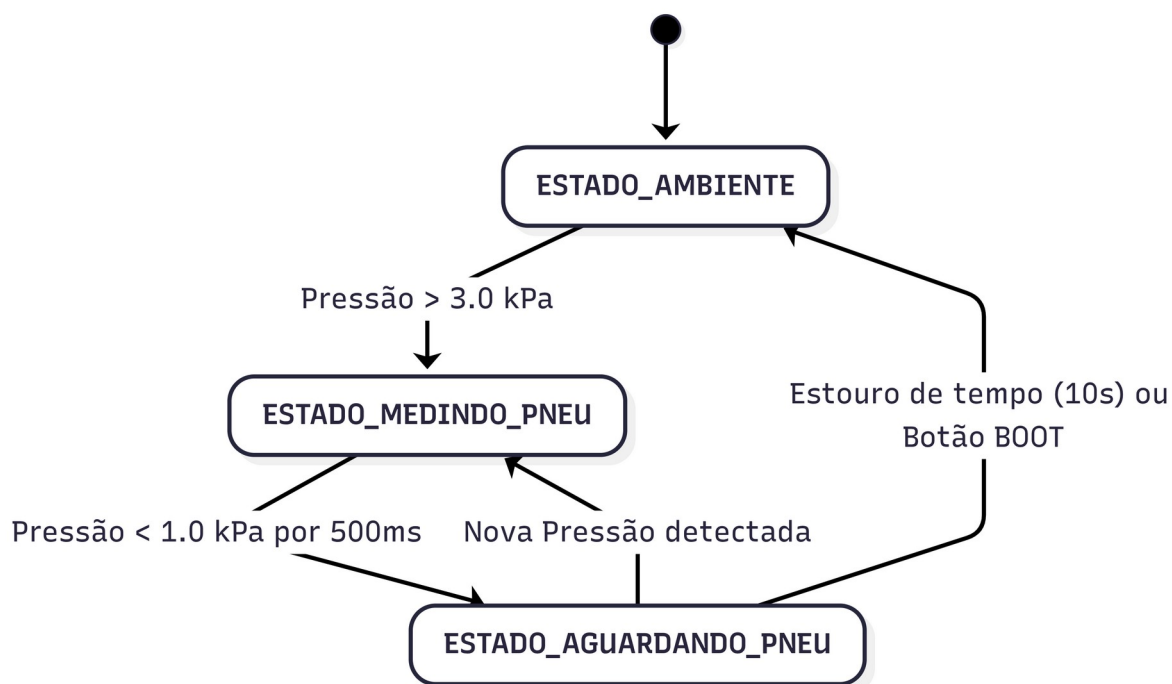


Imagem ilustrando o diagrama de máquina de estados

Ao ligar o dispositivo ativamos o que denominamos no projeto de ESTADO_AMBIENTE quando as interfaces disponíveis mostram apenas as leituras de Pressão e Temperatura de ambiente e permanecem em uma situação aguardando o gatilho para iniciar a leitura da pressão de Pneu.

Esse gatilho é disparado quando acontece uma variação de pressão no bico da mangueira a partir de 3.0 kPa, situação que inicia o que chamamos de ESTADO_MEDINDO_PNEU. Nessa situação apresenta-se na tela a leitura do PNEU índice 1.

É feita a leitura da pressão enquanto a variação é positiva e crescente, indicando que está entrando ar pelo bico da mangueira. A partir do momento que esse crescimento se interrompe e voltamos a ter uma leitura próxima da pressão ambiente (Pressão < 1.0 kPa por 500ms), consideramos essa como a leitura final daquele pneu e passamos para a próxima situação que chamamos de ESTADO_AGUARDANDO_PNEU.

Esse novo estado aguarda uma nova variação de pressão como descrito anteriormente e assim que acontece, incrementa-se o índice para PNEU2 e assim por diante.

O processo assim permanece por um tempo de retenção de 10s ou quando o Botão BOOT é pressionado, voltando a situação inicial ESTADO_AMBIENTE.

4 MONTAGEM DO SENSOR

A montagem física do sistema utiliza a placa TTGO ESP32 conectada a um protoboard para testes, na qual foram conectados os sensores por jumpers. Os sensores SMP3011 e BMP280 foram conectados em paralelo ao mesmo barramento físico I2C do microcontrolador, utilizando resistores de pull-up internos configurados via firmware.

O circuito conta ainda com um LED indicador conectado à GPIO 16 para sinalização visual de operação (pisca lento para ambiente, pisca médio para prontidão e acesso constante para medição ativa) e o botão de BOOT integrado à GPIO 0 operando com circuito de pull-up físico para o encerramento do ciclo.

Já o sensor SMP3011 foi acoplado provisoriamente para testes em uma tubulação de PVC formada por luva e dois niples ou acoplamentos, permitindo um bom encaixe do sensor, sem danificá-lo por pressionamento e garantindo uma boa vedação com uma borracha retentora e um pouco de veda-rosca.

O conjunto de tubulação era finalizado em sua extremidade com um bico de mangueira, permitindo que ao assoprar o cano simulávamos a conexão e desconexão de um pneumático.



Conectores hidráulicos utilizados

Cabe ressaltar que optamos por não priorizar um conector de bico para pneus, uma vez que conforme as orientações do instrutor, a prioridade era o código. Além disso, para testar seria necessário ter a disposição pneus cheios com pressões conhecidas para calibragem do dispositivo.

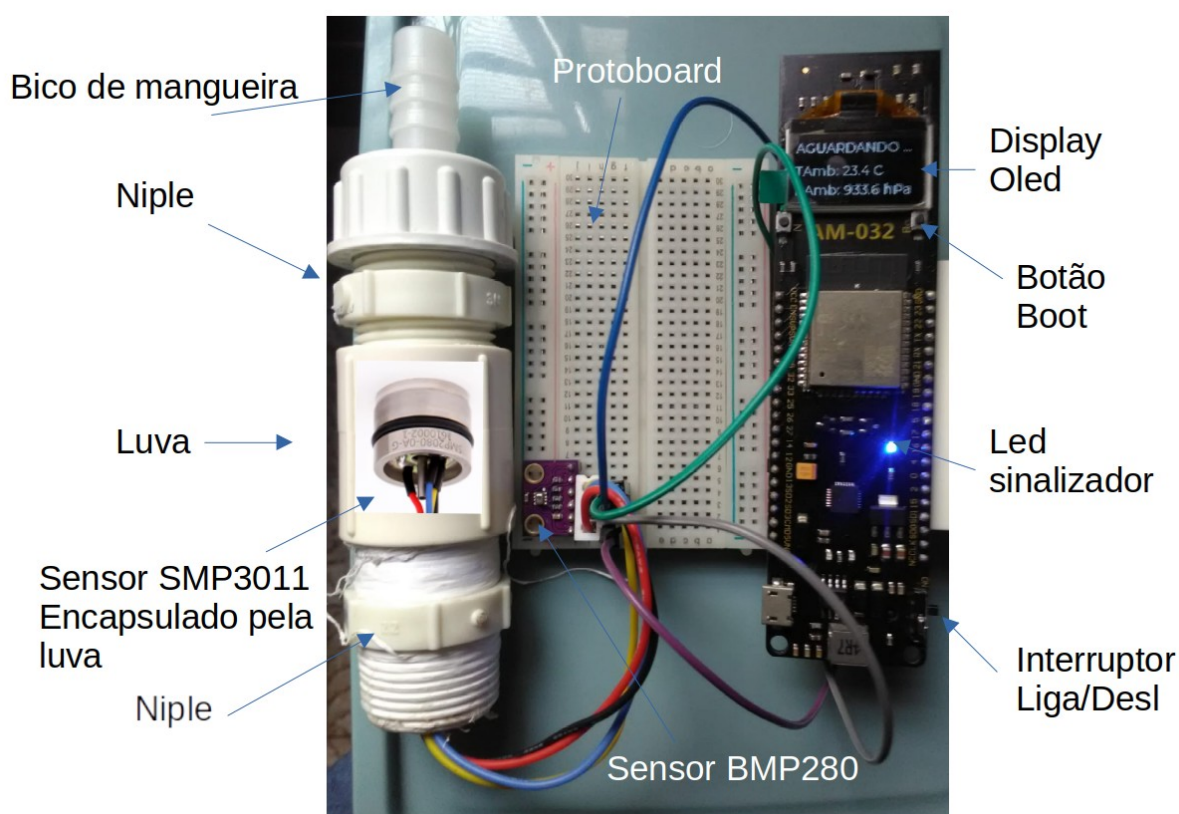


Imagem detalhando a montagem

Pedimos atenção à coordenação do curso para alertar que durante a disciplina **CONTROLE E AUTOMAÇÃO** ao longo de 2024-2, de onde esperávamos ter subsídios para o projeto, não houve qualquer menção a maioria sensores vistos ao longo de todo o curso de Sistemas Embarcados, pois o foco da matéria foi **TOTALMENTE** voltado a cálculos de motores

5 INTEGRAÇÃO COM SISTEMA SUPERVISÓRIO

Tendo em vista que o display utilizado para apresentar informações era limitado pelo seu diminuto tamanho, vamos considerar como Sistema Supervisório apenas a interface Front End no celular.

Sendo assim, podemos considerar o sistema supervisório deste projeto como descentralizado e embarcado diretamente no próprio microcontrolador através da pilha de protocolos de rede do ESP-IDF. Ao inicializar, a subsystem esp_netif e o driver Wi-Fi configuram o ESP32 no modo Access Point (AP) com criptografia aberta.

Foi implementada a biblioteca esp_http_server configurada com uma stack de 8KB para o tratamento estável de requisições. Quando um dispositivo móvel (smartphone ou tablet) se conecta à rede "PROJETO" e requisita o IP padrão 192.168.4.1, o manipulador de rota URI (root_get_handler) realiza a alocação dinâmica de memória, lê os arrays globais que armazenam as variáveis de pico dos sensores e renderiza uma página HTML dashboard responsiva. A página utiliza a diretiva http-refresh para atualizar os dados estruturados na tela do usuário a cada 2 segundos.



Tela exibida no celular com o dispositivo aguardando leitura

O código em C/C++ que roda no ESP32 manipula uma variável string que gera um texto de html escrito linearmente que, para compreensão do leitor vamos printá-lo aqui com indentação:

```
<!DOCTYPE html>
<html>
<head>
<meta charset='UTF-8'>
<meta http-equiv='refresh' content='2'>
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
<title>Manômetro Smart</title>
<style>
body { background:#121214; color:#e1e1e6;
font-family:sans-serif; padding:15px; margin:0; text-align:center; }
.container { max-width:480px; margin:0 auto; }
h1 { color:#04d361; font-size:24px; margin-bottom:5px; }
.status-badge { display:inline-block; padding:8px 15px;
border-radius:20px; font-weight:bold; color:#000; font-size:14px;
margin-bottom:15px; background:#28a745; }
.card { background:#202024; padding:15px; border-radius:8px;
margin-bottom:15px; border:1px solid #29292e; text-align:left; }
.card h3 { margin:0 0 10px 0; color:#8d8d99; font-size:14px;
text-transform:uppercase; letter-spacing:1px; }
.card p { margin:5px 0; font-size:18px; font-weight:bold; }
table { width:100%; border-collapse:collapse;
background:#202024; border-radius:8px; overflow:hidden; border:1px solid #29292e; }
th { background:#29292e; color:#04d361; padding:12px; font-size:14px; text-align:left; }
td { padding:12px; border-bottom:1px solid #29292e; font-size:16px; }
tr:last-child td { border-bottom:none; }.no-data { text-align:center;
color:#7c7c8a; padding:20px; font-style:italic; }
</style>
</head>
```



```

<body>
<div class='container'><h1>☐ Manômetro Inteligente</h1>
  <div class='status-badge'>AGUARDANDO</div>
</div>
<div class='card'><h3>🌡️ Condições Ambientais (BMP280)</h3>
  <p>Temperatura: <span style='color:#ff8b6c;'>23.9 °C</span></p>
  <p>Pressão Atmosférica: <span style='color:#8be9fd;'>933.6 hPa</span></p>
</div>
<div class='container' style='text-align:left; margin-bottom:10px;'>
  <h3 style='color:#8d8d99; font-size:14px; text-transform:uppercase; margin:0;'>
    📜 Histórico de Pneus Medidos</h3>
  </div>
<table>
  <thead><tr><th>Pneu</th><th>Pressão (PSI)</th><th>Temperatura</th></tr>
  </thead>
  <tbody><tr><td colspan='3' class='no-data'>Nenhum pneu medido nesta sessão.</td></tr>
  </tbody>
</table>
</body>
</html>

```

Prints do código html resultante

6 COMUNICAÇÃO ENTRE APLICAÇÃO E USUÁRIO

Conforme já dissemos em parágrafos anteriores, a interface homem-máquina (IHM) é dividida em dois níveis de visualização:

6.1 INTERFACE LOCAL (DISPLAY OLED SSD1306)

Gerenciada pela biblioteca LVGL assíncrona. Em repouso, exibe o status de sistema aguardando e as variáveis de calibração do BMP280. Em operação, exibe o pneu sob análise (Ex: P1) e congela o valor final por 10 segundos, oferecendo uma leitura clara direto no hardware. A sincronização gráfica impede piscadas (flicker) na tela.



Display Oled

6.2 INTERFACE REMOTA (WEB BROWSER)

Destinada ao acompanhamento gerencial. Apresenta um badge colorido de status em tempo real (Verde para Repouso, Amarelo para Transição, Vermelho para Medição Crítica) e monta uma tabela dinâmica contendo o histórico imutável de todos os pneus avaliados na sessão atual. Os dados são retidos na memória RAM da aplicação e só são expurgados mediante comando deliberado do usuário através do botão físico.



Tela do celular exibindo leituras

7 CRIAÇÃO DE UM REPOSITÓRIO NO GITHUB

O código original compartilhado durante o treinamento da Tex foi clonado a partir de repositório da empresa no GitHub em uma pasta local de nossa máquina com sistema MX Linux com o programa GIT instalado, através do comando:

```
git clone https://github.com/eng-software/TreinamentoESP32_Medidor.git MUTEX --recursive -b MUTEX
```

Localmente, iniciamos o GIT, removemos o vínculo com o repositório original da TEX, criamos um branch principal, fizemos nossas alterações no código e promovemos os nossos primeiro push. Para isso foram utilizados os comandos:

```
git remote remove origin
```

```
git remote add origin https://github.com/jukaribeiro/Projeto-Integrador-3
```

Após realizar as primeiras modificações no código utilizamos:

```
git add .
```

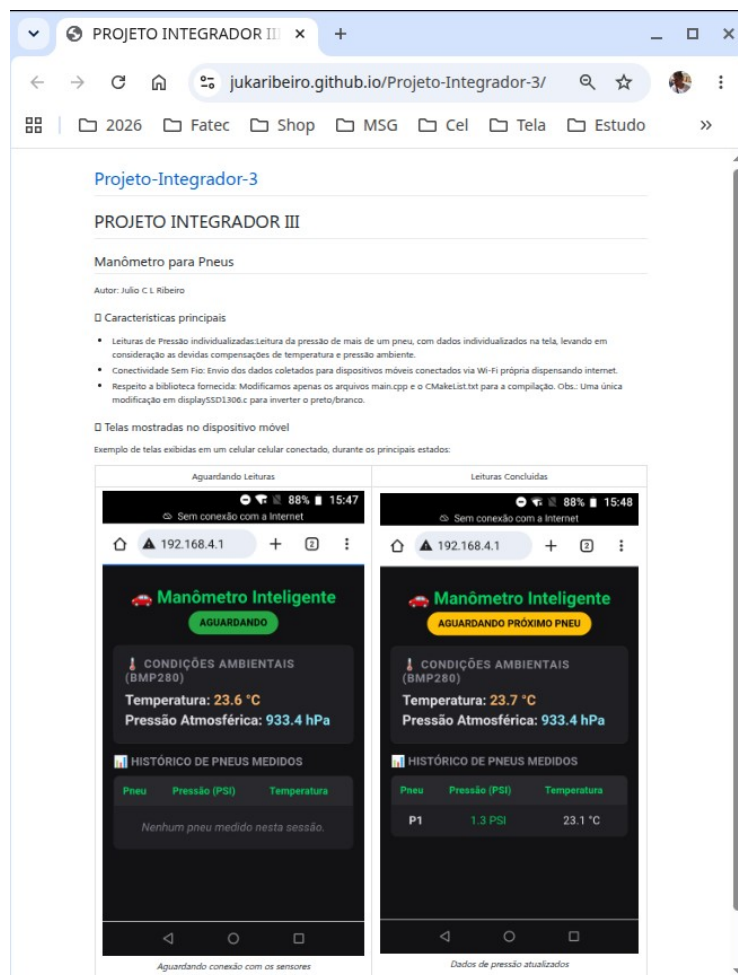
```
git commit -m "Primeiro commit"
```

```
git branch -M main
```

```
git push -u origin main
```

Para sermos honestos fizemos poucos commits e pushes para não vaziar o código antes do final do projeto. A maior parte das atualizações foram para a criação do arquivo README.MD para apresentação do projeto para os representantes da TEX.

Posteriormente, configuramos nosso repositório para mostrar o arquivo README como uma página no GitHub Pages em: <https://jucaribeiro.github.io/Projeto-Integrador-3/>



Site criado no Github pages

8 CONCLUSÃO

O desenvolvimento do manômetro inteligente sob a estrutura nativa ESP-IDF demonstrou a viabilidade de criar soluções de instrumentação portáteis de alta confiabilidade. A substituição de lógicas de varredura simples por uma arquitetura preemptiva baseada em tarefas e máquina de estados eliminou falhas críticas de concorrência e vazamento de memória comumente associados ao uso de displays e servidores web simultâneos.

O protótipo final atingiu todos os requisitos de engenharia propostos: apresentou sensibilidade adequada para as transições automáticas de fluxo, precisão na retenção temporária de dados locais para leitura e total estabilidade na distribuição remota do histórico de medições via Wi-Fi.

Na medida do possível, foram aplicados os conceitos aprendidos nas diversas disciplinas ministradas no semestre.

9 REFERÊNCIAS BIBLIOGRÁFICAS

BLOG ELETROGATE. **FreeRTOS – Task, Trabalhando com Multitarefa**s. Disponível em: < <https://blog.eletrogate.com/freertos-task-trabalhando-com-multitarefa/> >. Acesso em 28 nov 2024.

CASA DO DESENVOLVEDOR. **Requisitos funcionais e não funcionais: o que são e como identificar?**. Disponível em: < <https://blog.casadodesenvolvedor.com.br/requisitos-funcionais-e-nao-funcionais/> >. Acesso em 28 nov 2024.

CRESCER ENG. **Data e hora no ESP32 sem RTC** Disponível em: <<https://www.crescerengenharia.com/post/hora-esp32-rtc> > Acesso em 28 nov 2024.

DEVMEDIA. **Introdução a Requisitos de Software**. Disponível em: < <https://www.devmedia.com.br/introducao-a-requisitos-de-software/29580> >. Acesso em 20 nov 2024.

ESPRESSIF. **ESP-IDF Programming Guide** Disponível em: <<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>> Acesso em 28 nov 2024.

MONITORATEC. **Entenda a especificação de requisitos de software em projetos**. Disponível em: < <https://www.monitoratec.com.br/blog/especificacao-de-requisitos-de-software/> >. Acesso em 20 nov 2024.